

Basic Operators in Python

Operators are special symbols that perform operations on variables and values. Python supports the following basic types of operators.

1. Arithmetic Operators

Operator	Name	Description	Example	Result
+	Addition	Adds two values	$a = 10, b = 3$	13
-	Subtraction	Subtracts second value from first	$a - b$	7
*	Multiplication	Multiplies two values	$a * b$	30
/	Division	Divides first value by second (float result)	a / b	3.333...
//	Floor Division	Divides and returns integer part	$a // b$	3
%	Modulus	Returns remainder	$a \% b$	1
**	Exponentiation	Returns a raised to the power b	$a ** b$	1000

Example :

```
a = 10
b = 3
a + b = 13
a - b = 7
a * b = 30
a / b = 3.333...
a // b = 3
a % b = 1
a ** b = 1000
```

2. Assignment Operators

Operator	Example	Same As
=	$x = 5$	$x = 5$
+=	$x += 3$	$x = x + 3$
-=	$x -= 3$	$x = x - 3$
*=	$x *= 3$	$x = x * 3$
/=	$x /= 3$	$x = x / 3$
//=	$x //= 3$	$x = x // 3$
%=	$x \% = 3$	$x = x \% 3$
**=	$x ** = 3$	$x = x ** 3$

Example :

```
x = 10
x += 5 # x = x + 5 → 15
x -= 3 # x = x - 3 → 12
x *= 2 # x = x * 2 → 24
x /= 4 # x = x / 4 → 6.0
x //= 2 # x = x // 2 → 3
x %= 2 # x = x % 2 → 1
x **= 3 # x = x ** 3 → 1
```

3. Comparison (Relational) Operators

Operator	Name	Example	Result
==	Equal to	$5 == 5$	True
!=	Not equal to	$5 != 3$	True
>	Greater than	$5 > 3$	True
<	Less than	$5 < 3$	False
>=	Greater than or equal to	$5 >= 3$	True
<=	Less than or equal to	$5 <= 3$	False

Example :

```
a = 8, b = 4
a == b → False
a != b → True
a > b → True
a < b → False
a >= b → True
a <= b → False
```

4. Logical Operators

Operator	Name	Description	Example	Result
and	Logical AND	True if both conditions are True	$(5 > 3) \text{ and } (2 < 4)$	True
or	Logical OR	True if at least one condition is True	$(5 > 3) \text{ or } (2 > 4)$	True
not	Logical NOT	Reverses the result	$\text{not } (5 > 3)$	False

Example :

```
a = 5, b = 3
(a > b) and (a < 10) → True
(a > b) or (a < 0) → True
not (a > b) → False
```

5. Membership Operators

Operator	Description	Example	Result
in	Returns True if value is found in sequence	'a' in 'apple'	True
not in	Returns True if value is not found in sequence	'z' not in 'apple'	True

Example :

```
s = "python"
'p' in s → True
'y' in s → True
'x' not in s → True
```

6. Identity Operators

Operator	Description	Example
is	True if both variables refer to same object	$a \text{ is } b$
is not	True if both variables do not refer to same object	$a \text{ is not } b$

Example :

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
a is b → True
a is c → False
a is not c → True
```

★ Note :

- Operator precedence decides the order in which operations are performed.
- You can use parentheses () to change the order of evaluation.

Example :

```
result = 5 + 3 * 2 # 3*2 happens first → 11
result = (5 + 3) * 2 # parentheses first → 16
```

String Operations in Python

A string is a sequence of characters enclosed in single quotes (''). double quotes (") or triple quotes (''' or """).

Example : `s = "Hello, Python!"`

1. Accessing Characters (Indexing and Slicing)

Indexing : Each character in a string has an <u>index</u> . Index starts from 0.		Slicing : Used to get a part of string. Syntax : <code>s[start : end : step]</code> (end is exclusive)
<code>s = "Python"</code>	Examples :	Examples :
Index : 0 1 2 3 4 5	<code>s[0]</code> → 'P' (first character)	<code>s[0:3]</code> → 'Pyt' (start to end-1)
Char : P y t h o n	<code>s[3]</code> → 'h' (character at index 3)	<code>s[2:5]</code> → 'tho'
	<code>s[-1]</code> → 'n' (last character)	<code>s[:4]</code> → 'Pyth' (from start)
	<code>s[-2]</code> → 'o' (second last character)	<code>s[3:]</code> → 'hon' (to end)
		<code>s[::2]</code> → 'Pto' (step = 2)
		<code>s[::-1]</code> → 'nohtyP' (reverse string)

2. Common String Operations / Methods

S.No.	Operation / Method	Description	Example	Output
1.	<code>len(s)</code>	Returns length of string	<code>len("Python")</code>	6
2.	<code>s.upper()</code>	Converts string to uppercase	<code>"python".upper()</code>	'PYTHON'
3.	<code>s.lower()</code>	Converts string to lowercase	<code>"PYTHON".lower()</code>	'python'
4.	<code>s.title()</code>	Converts first character of each word to uppercase	<code>"hello world".title()</code>	'Hello World'
5.	<code>s.capitalize()</code>	Converts first character to uppercase	<code>"python programming".capitalize()</code>	'Python programming'
6.	<code>s.strip()</code>	Removes leading and trailing spaces	<code>" hello ".strip()</code>	'hello'
7.	<code>s.lstrip()</code>	Removes leading spaces	<code>" hello".lstrip()</code>	'hello'
8.	<code>s.rstrip()</code>	Removes trailing spaces	<code>"hello ".rstrip()</code>	'hello'
9.	<code>s.replace(old, new)</code>	Replaces all occurrences of old with new	<code>"banana".replace('a', 'o')</code>	'bonono'
10.	<code>s.split(sep)</code>	Splits string into list by separator	<code>"a,b,c".split(',')</code>	['a', 'b', 'c']
11.	<code>s.join(iterable)</code>	Joins elements of iterable with the string	<code>"-".join(['a', 'b', 'c'])</code>	'a-b-c'
12.	<code>s.find(sub)</code>	Returns first index of substring (-1 if not found)	<code>"hello".find('e')</code>	1
13.	<code>s.count(sub)</code>	Returns number of occurrences of substring	<code>"banana".count('a')</code>	3
14.	<code>s.startswith(sub)</code>	Checks if string starts with substring	<code>"Python".startswith('Py')</code>	True
15.	<code>s.endswith(sub)</code>	Checks if string ends with substring	<code>"Python".endswith('on')</code>	True

3. More Examples

<code>s = " Learn Python Programming "</code>	
<code>print(s)</code>	→ " Learn Python Programming "
<code>print(s.strip())</code>	→ "Learn Python Programming"
<code>print(s.lower())</code>	→ " learn python programming "
<code>print(s.upper())</code>	→ " LEARN PYTHON PROGRAMMING "
<code>print(s.replace("Python", "Java"))</code>	→ " Learn Java Programming "
<code>print(s.split())</code>	→ ['Learn', 'Python', 'Programming']
<code>print("-".join(s.split()))</code>	→ "Learn-Python-Programming"

★ Notes :

- Strings are immutable (cannot be changed).
- Operations create a new string and original string remains same.
- Index out of range will give `IndexError`.

★ **Summary :** Strings in Python support many built-in operations and methods that make text processing easy and powerful.

